Physics Attestation Layer

A physics-based digital twin for robot arm monitoring

Project overview for readers new to the project:
what it is, why it matters, how it works, what has been achieved

Technical overview · November 2026

Franka Panda · NVIDIA Isaac Sim · Pinocchio rigid-body dynamics

Contents

1. Executive summary 3

2. Background: why robot arms need to be watched 3

2.1 What an industrial arm is . . . 3

2.2 What goes wrong in production . . . 4

2.3 The physics-based alternative . . . 4

3. The core concept: the residual 4

3.1 Which torque? Applied versus measured . . . 5

3.2 The speed regime . . . 5

4. Project objectives 5

5. System architecture 6

5.1 The simulator (the digital twin) . . . 7

5.2 The analysis . . . 7

5.3 The dashboard . . . 7

6. How the simulator works 7

6.1 Torque control . . . 7

6.2 Realistic imperfections . . . 7

6.3 Motion generation: from a task to joint trajectories . . . 8

6.4 The gripper and the cube: a real pick . . . 8

6.5 Fault and disturbance injection . . . 9

6.6 Faithful windowed operation . . . 9

7. What the data look like 9

8. How the analysis works 10

8.1 From telemetry to residual . . . 10

8.2 Confidence: when can the arm be trusted to reveal a force? . . . 10

8.3 Payload estimation . . . 10

8.4 Fault detection . . . 10

8.5 A finding: non-repeating tasks need a healthy reference . . . 11

9. The dashboard 11

9.1 What is on the screen . . . 11

9.2 The three data sources . . . 12

9.3 What happens when you press Run pick . . . 12

9.4 Reading a result: three scenarios . . . 12

10. How we know the work is correct 13

11. What has been achieved 13

11.1 Capabilities delivered . . . 13

11.2 Results . . . 14

12. Limitations and open problems 15

13. Industrial applications and importance 15

13.1 Why this matters . . . 16

13.2 Applications . . . 16

13.3 Why a digital twin first . . . 16

13.4 Path to deployment . . . 16

14. Using the system 17

15. Glossary 17

1. Executive summary

This document describes a research and engineering project that builds a **physics-based digital twin of an industrial robot arm** and uses it to answer, continuously and automatically, two questions that matter to anyone who runs robots in production: *Is the arm behaving the way physics says it should?* and *What is it carrying?*

The project is called the Physics Attestation Layer. The word *attestation* is deliberate. Instead of trusting a robot's own status messages, the layer independently checks the arm's behaviour against the laws of mechanics. A mathematical model of the arm predicts the torque each joint needs to produce the motion that was measured. If the torque that the controller actually applied differs from that prediction, something physical that the model does not know about is happening: an unexpected load, a joint with extra friction, a seized bearing, a communication delay. That difference is called the **residual**, and almost everything in the project is a way of producing a clean residual, understanding it, and acting on it.

The work is split into two halves. The **simulation side** builds a high-fidelity virtual Franka Panda arm in NVIDIA Isaac Sim, runs it under realistic torque control, performs real pick-and-place tasks with a physical gripper and cube, injects faults on demand, and produces telemetry. The **analysis side** turns telemetry into a residual, a measure of how trustworthy the residual is at the current arm pose, an estimate of the carried mass, and a fault detector. A web dashboard ties both together: the engineer sees the simulator's live picture next to the residual and the alerts, and can start picks, change the payload and inject faults with a click.

The simulation side is complete and has been checked extensively. Payload estimation is accurate to a few percent when the load is known to the physics; faults are detected within roughly 0.03 to 0.12 seconds of onset in most cases. Several honest limitations remain, the largest being that the payload estimate on a real, contact-held 2 kg object is 25 to 36 percent low for reasons not yet understood. These are set out plainly in Sections 11 and 12.

> **In one sentence**
> A physics model of a robot arm is used as a reference witness: whatever the arm does that the model cannot explain is measured, localised to a joint, and turned into a payload estimate or a fault alert.

2. Background: why robot arms need to be watched

2.1 What an industrial arm is

A serial robot arm is a chain of rigid links connected by motorised joints. The arm used in this project, the Franka Panda, has seven revolute joints, a reach of roughly 0.85 m, a rated payload of about 3 kg, and a two-finger parallel gripper. Each joint contains a motor, a gear reducer and a position sensor, and reports a torque estimate. The first four joints are large and strong (rated for roughly 87 N·m); the last three are small wrist joints (about 12 N·m). Seven joints give the arm one redundant degree of freedom, which is useful for avoiding obstacles but means that the same hand position can be reached with many different joint configurations.

Motion is commanded either as positions (the joint controller tries to reach given angles) or as torques (the application computes the exact torque each motor should produce). Torque control is how modern collaborative and research arms achieve compliant, safe and precise behaviour, and it is the mode used throughout this project because it exposes the quantity that physics reasoning needs.

2.2 What goes wrong in production

A robot cell can degrade or fail quietly. The following are common and costly:

- **Wear and lubrication loss** raise joint friction gradually. The arm still moves, but consumes more energy, heats up, loses accuracy and eventually fails.
- **Seized or sticking joints** from contamination, brake faults or mechanical damage.
- **Wrong or unexpected payloads**: a gripper picks up a part that is heavier than specified, two parts instead of one, a tool that was not removed, or an object that slips and is dropped. A mis-declared payload degrades control accuracy and can exceed safe limits.
- **Communication or control delays** from network congestion or software load, which turn a stable controller into an oscillating one.
- **Backlash** (play) in gears, which reduces positioning accuracy and causes impacts.

Conventional condition monitoring relies on thresholds on motor current, temperature or vibration. These are blunt: the same current can mean a heavy payload, a fast motion, or a failing bearing. Without a model of what the current *should* be, the monitor cannot tell the difference, so it either misses problems or raises false alarms.

2.3 The physics-based alternative

The motion of a rigid-body arm is described precisely by well-established equations. Given the joint positions, velocities and accelerations, the torque each joint must supply equals the sum of four contributions: an **inertial** term (accelerating the links), a **velocity-coupling** term (Coriolis and centrifugal effects between joints), a **gravity** term, and **friction**. All of these can be computed from the arm's mass distribution and geometry. Compare that computed torque with the torque actually applied, and the difference isolates exactly the effects the model omits. That is the idea behind the whole project.

3. The core concept: the residual

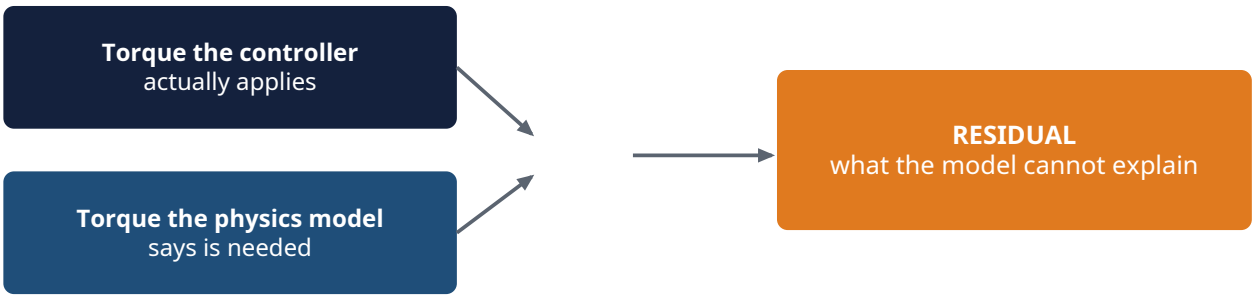


Figure 1 · The residual is the applied torque minus the torque the physics model predicts for the measured motion.

For each joint and at each instant, the project computes the residual as the difference between the torque the controller applied and the torque the physics model predicts is necessary to

produce the observed position, velocity and acceleration. If the model is accurate and nothing unusual is happening, the residual is close to zero apart from sensor noise. Anything physical that is not in the model shows up there, and its **shape** tells what it is:

Cause	What the residual looks like	Why
Extra mass held by the gripper	A smooth, pose-dependent torque on the joints that carry the load; largest where the arm is stretched out	A mass at the hand adds a weight whose lever arm depends on the arm's geometry; it is also felt as extra inertia when accelerating
Increased friction on one joint	A torque on that joint that opposes its motion and changes sign when it reverses	Friction always acts against velocity
Locked or seized joint	Large, growing torque on that joint, and a joint that stops following its command	The controller keeps pushing while the joint cannot move
Command delay	Oscillating tracking error and residual on all joints, growing with speed	The controller reacts to stale information
Backlash	Small, impulsive disturbances at motion reversals	Gears lose and re-establish contact

Two subtleties shape the design of the entire system, and understanding them is essential for interpreting results.

3.1 Which torque? Applied versus measured

A joint torque sensor in the simulator reports the *net* torque acting at the joint. Friction, a seized joint and gear play do not appear in that reading, because they act inside the joint, between the motor and the sensor, or are balanced by it. The torque that the controller *commands* does contain their effect, because the controller has to push harder to overcome them. The project therefore uses the **applied (commanded) torque** as the signal for fault detection, while the **measured** torque is still the right signal for payload and external loads, which do appear in it. This distinction was established by direct measurement and recorded in every run.

3.2 The speed regime

The residual is only trustworthy while the arm moves at moderate speed. Above about half of each joint's velocity limit, effects that are hard to model (drive dynamics, solver artefacts) dominate the residual. All detection experiments are therefore run in a regime of at most 50 percent of the velocity limit, and every run records the speed it actually reached. This is a stated scope decision of the project, not an oversight: claims are made only inside this regime.

4. Project objectives

The project is organised around seven objectives. Each one produces a distinct capability and each has measurable acceptance criteria.

#	Objective	What it delivers	Status
O1	Telemetry	A well-defined, validated stream of joint positions, velocities, torques, model terms and hand pose at 1 kHz, with the conventions (timing, units, torque meaning) documented	Complete
O2	Residual	A clean, filtered residual per joint, computed from an independent physics model of the arm	Complete
O3	Confidence	A 0-to-1 score of how well-conditioned the arm pose is for inferring forces and mass, and a gate that pauses estimates at poorly conditioned poses	Complete; tested on simulated poses
O4	Payload estimation	An online estimate of the mass carried by the gripper, with an uncertainty band	Complete; weak case on a real 2 kg object (Section 11)
O5	Fault detection	Detection and localisation of friction increase, locked joint, delay and other arm faults, with low false-alarm rate	Complete for the covered faults; some faults undetectable (Section 12)
O6	Trajectory benchmark	Comparison of three motion styles on cycle time, peak torque and energy, including on a full pick cycle	Complete
O7	Presentation	A live dashboard showing the simulator, residual, estimates and alerts, with controls	Dashboard and live link complete; in-simulator overlay not built

A deliberate scope decision
The project treats disturbances as **arm faults and load changes**, not as attacks by an adversary. Nobody is manipulating object positions or tampering with signals; what changes are friction values, joint health, communication delay, and the mass being carried. The question is how the arm copes and how it moves.

5. System architecture

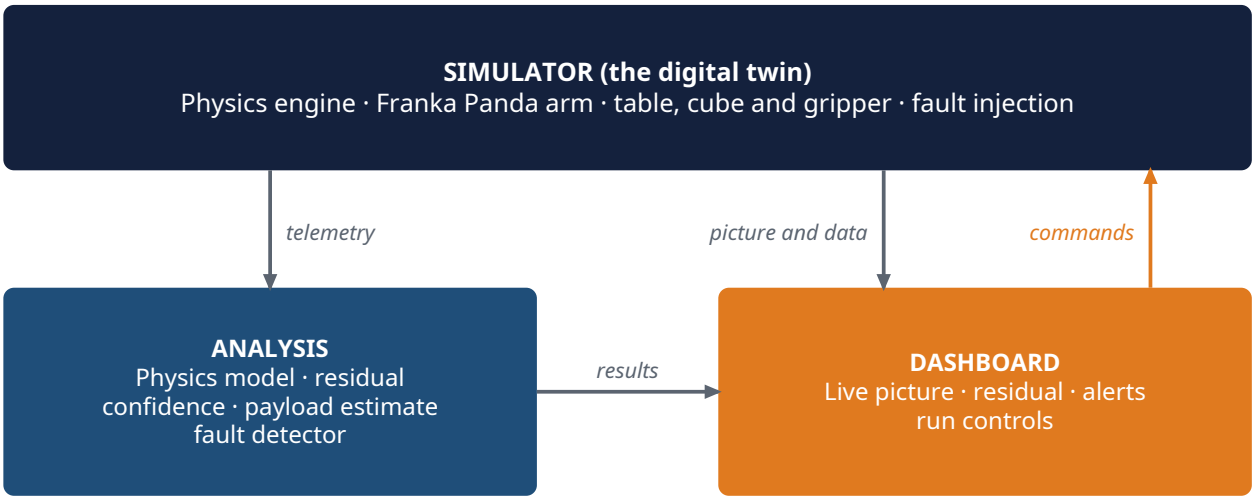


Figure 2 · The three parts of the system and the data that flows between them.

The system has three parts that communicate over simple network interfaces.

5.1 The simulator (the digital twin)

The simulator is built on NVIDIA Isaac Sim, a GPU-accelerated robotics simulator with a rigid-body physics engine (PhysX) and a physically based renderer. It contains a faithful model of the Franka Panda: link masses, inertia tensors, joint limits, torque limits, gear armature and friction, plus a table, a rigid cube, and the arm's two-finger gripper with realistic contact. Physics advances at 1,000 steps per second; telemetry is recorded or streamed from the same clock.

It is called a *twin* in the sense that it is the stand-in for the real machine that provides ground truth: the simulator knows exactly which fault was injected and exactly how heavy the cube is, which a real plant never reveals. This ground truth is what lets every claim in the analysis be checked.

5.2 The analysis

The analysis side contains an **independent** physics model of the arm built with Pinocchio, an open-source rigid-body dynamics library widely used in robotics research. Independence matters: the model is built from the arm's published description, not from the simulator's internal state, so agreement between the two is meaningful evidence rather than a tautology. Around this model sit the residual computation, the confidence measure, the payload estimator and the fault detector (Section 8).

5.3 The dashboard

The dashboard is a web application that presents all of the above to an operator and sends commands back to the simulator. It is described in Section 9.

6. How the simulator works

6.1 Torque control

The arm is driven by a computed-torque controller. At each millisecond the controller reads the joint positions and velocities, looks up the desired position, velocity and acceleration from the trajectory, and computes the torque that would produce the desired acceleration plus a correction proportional to the position and velocity errors. The controller uses the simulator's own mass, Coriolis and gravity terms for its feed-forward, so it is, deliberately, a *perfect-knowledge* controller. Any load or fault the controller does not know about therefore shows up as the difference between what it applied and what the nominal model predicts, which is exactly what the residual measures.

6.2 Realistic imperfections

A perfect simulation would make detection trivially easy and the results meaningless. Several imperfections are therefore added deliberately:

- **Model mismatch.** The simulated link masses differ from the analysis model's by 5 percent, representing the fact that no real arm matches its datasheet.
- **Friction.** Each joint has static, dynamic and viscous friction expressed in physical torque units. (An early version used a dimensionless coefficient, which was a bug that was found and fixed.)
- **Sensor noise.** Joint position, velocity and torque readings receive random noise of representative magnitude (about 0.00002 rad, 0.001 rad/s and 0.02 N·m), with a recorded

random seed so that runs are repeatable and false-alarm *rates* can be measured over many healthy runs.

- **Inertia authoring.** The full inertia tensor of each link is written into the physics engine, not only its diagonal. An earlier convention error produced a 0.73 relative error in the mass matrix; after correction it is 0.00004 against an independent calculation.

6.3 Motion generation: from a task to joint trajectories

Real tasks are described in Cartesian space: *put the fingertips above the cube, descend, grasp, lift, carry to another point, lower, release, retreat.* The controller, however, needs joint-space trajectories. The bridge is **inverse kinematics** (IK): given a desired hand position and orientation, find joint angles that achieve it. For a seven-joint arm this has many solutions; the project solves it numerically with a damped least-squares method that stays close to the previous solution, biases the redundant joint towards a comfortable posture, keeps the gripper vertical, respects joint limits, reports unreachable targets, and refuses to switch to a different elbow configuration mid-task.



Figure 3 · The same chain serves every task, style and payload.

Between the IK waypoints, the arm moves according to one of three **trajectory styles**:

- **Trapezoidal**: constant acceleration, cruise, constant deceleration. Fast and simple, but acceleration changes abruptly.
- **S-curve**: limits the rate of change of acceleration (jerk), giving smoother motion and less vibration.
- **Torque-aware**: a smooth polynomial profile whose duration is chosen from the size of the move so that peak torque stays low.

Because the chain ends in the same torque controller, one inverse-kinematics solution serves all styles and all payloads; only the speed and acceleration limits belong to the style. Continuous paths such as an ellipse are produced by solving IK at every millisecond along the curve.

6.4 The gripper and the cube: a real pick

An early version of the pick was not a pick at all: the arm followed a joint-space sine wave, the fingers never moved, the payload was an invisible mass added to the hand, and the cube in the window was a drawing that followed the hand. It was a calibration motion with a story attached. The current version is physical:

	Modelled pick	Physical pick
Payload	A point mass written into the hand's inertia at the grasp and removed at the release	A real rigid cube on a real table, held by the fingers
Gripper	Not driven	Both fingers driven: open to approach, closed to grasp, open to release

	Modelled pick	Physical pick
Where the weight appears	Inside the arm's own mass and gravity terms	Only through contact forces on the fingers, exactly as in a real cell
Use	Exact known mass: ground truth for the estimator	Realism: tests the estimator on a load it was not told about

The gripper uses a position-driven finger pair with measured grip force of about 33 N per finger, high-friction pads, contact offsets tuned for stable stacking and a solver configured for fast contact. The recipe was adapted from an earlier project that uses the same arm and gripper, then verified by measurement here. Two facts about the asset were found by measurement rather than assumed: the second finger has no drive until explicitly configured, and the force limit is shared by the two fingers, so it must be set to twice the per-finger value. The grasp itself is open-loop and timed: the fingers close at a scheduled moment with no force or slip feedback.

6.5 Fault and disturbance injection

A fault injector can modify the running plant at a chosen moment on a chosen joint:

Fault	How it is injected	Typical parameter
Increased friction	Multiplies a joint's friction torque	2× to 8×
Locked / stuck joint	Applies a large braking friction so the joint cannot move	Seized joint
Command delay	Delays the torque command by a fixed time	50 ms
Backlash	Introduces a dead zone between motor and link	Small angle of play
Payload change	Adds or removes mass at the hand, or changes the cube's mass	0.2 to 3 kg
External push	Applies a known force at the hand (used for validation)	Known newtons

6.6 Faithful windowed operation

Rendering the simulation to a window exposed a subtle bug: the renderer's default interval advanced the simulation by 33 physics steps for each drawn frame, so the arm moved 33 times as far on every rendered data row while the logger counted a single step. Dynamics checks failed during motion yet passed at rest, which is how the defect was identified. The rendering interval is now equal to the physics step; the per-row motion ratio went from 33.3 to 1.00. Windowed and headless runs agree to about 0.04 N·m in torque, not bit for bit.

7. What the data look like

Every simulated run is stored together with a manifest describing exactly how it was produced: the physics rate and solver settings, the arm model, the control law and gains, the friction values, the injected fault (type, joint, size, onset time), the payload schedule, the noise seed, the maximum speed reached as a fraction of the limit, and flags such as whether the stored torque contains friction. Rows contain joint positions, velocities, commanded and measured torques, the simulator's own gravity and Coriolis terms, the mass matrix, the Jacobian, the hand pose, finger positions and forces, and the labels of what was injected.

Data are organised in suites, one per objective, so any objective's tests can be re-run alone or together. In total about 200 runs exist, including healthy runs with different noise seeds, matched healthy-and-faulty pairs sharing a seed so that the only difference is the fault, noise-free twins of key faults, faults at higher speed, held payloads at several poses, and the pick-and-place runs. Every run passes an automatic physical self-check before it is accepted, covering timing, torque limits, energy consistency, the match between the simulator's own dynamics and the independent model, and speed regime.

8. How the analysis works

8.1 From telemetry to residual

The independent model computes, from the measured positions, velocities and acceleration, the torque required. Acceleration is not measured; it is obtained by differencing velocity, which amplifies noise, so signals are passed through a zero-phase low-pass filter (a few tens of hertz bandwidth) before the comparison, and the first and last fraction of a second of each window, where the filter is unreliable, are discarded. The timing convention is crucial: the torque recorded for a row must be matched to the acceleration produced *by* that torque, not the one before it. This was verified experimentally; the evaluation point (before, during or after the step) changes the residual by less than a percent.

8.2 Confidence: when can the arm be trusted to reveal a force?

Not all arm poses are equally informative. Near a **singular pose**, such as the elbow fully extended, small errors in joint torque correspond to enormous errors in the force or mass inferred at the hand, because the arm's geometry cannot resolve force in some directions. The confidence measure quantifies this by the conditioning of the arm's Jacobian: it computes how much torque noise is amplified into hand force noise and into vertical force noise. If either exceeds a limit, the estimate is paused rather than allowed to be wrong.

Tested on simulated poses, the gate closes only at the extended-elbow singular pose, is open at the other probe poses (where a 1 kg load reads 0.993 to 1.003 kg), and pauses the estimate for about 44 percent of a pass through the singularity. A near-singular wrist pose is not gated, because its noise gains are well within the limit.

8.3 Payload estimation

A mass held at the hand contributes a known pattern of joint torques: the weight of the mass, projected through the arm's geometry, plus the force needed to accelerate it. Because the pattern is known per kilogram, the mass can be fitted to the residual by least squares, and averaged over time to reduce noise. The estimator subtracts a *tare* (its own reading on the empty arm) so that small constant model errors do not appear as phantom mass, and reports an uncertainty band derived from the scatter of recent estimates. Estimation is skipped at poses where the confidence gate is closed and during motion that is too fast for the residual to be valid.

8.4 Fault detection

The detector watches two channels: the residual itself and the *tracking error* (commanded minus measured position). Its logic is a statistical process-control scheme:

- **Calibration.** During the first three seconds the detector learns the normal mean and spread of each joint's signal, which captures noise and constant model error.
- **Threshold.** After calibration, a joint is flagged when its smoothed signal departs from normal by more than eight standard deviations.
- **Persistence.** The condition must hold for at least 50 milliseconds, rejecting noise spikes.
- **Pose gate.** No alarm is raised where the confidence measure says the pose is unreliable.
- **Localisation.** The first joint to alarm, and the channel it alarms on, indicate where the problem is.

Tuning an alarm threshold is a trade between missing real faults and crying wolf. Because simulated noise is seeded, many healthy runs can be scored, which gives a measured false-alarm rate rather than a guess.

8.5 A finding: non-repeating tasks need a healthy reference

When the arm performs a pick, the default calibration (the first three seconds) fails. A healthy pick produces a false alarm, because those first seconds are the unloaded approach and never include the carry or the place, so what is later normal looks abnormal. The obvious remedy, subtracting the estimated payload torque, turned out **not** to remove the false alarm, which proved the cause was elsewhere. The remedy that works is to calibrate against a **healthy run of the same task**, subtracting its residual and its tracking error from the run under test. With that reference, every injected fault is found on the correct joint. The practical implication is important for any real deployment: a task that does not repeat identically needs a recorded healthy baseline of that task.

9. The dashboard

The dashboard is the operator's window into the system. It is a single web page, opened in an ordinary browser, designed so that someone who has never seen the project can start a task, watch it, and understand what the analysis is saying. In its live mode it is connected to the simulator over the network and shows the simulator's own rendered picture.

9.1 What is on the screen

Area	What it shows	Why it matters
Header	Which data source is active (live simulator, stand-in model, or a replay of a recorded run), simulation time, alert count, pause and demo controls	Always makes clear whether the viewer is looking at the real simulator or a stand-in
Arm view	Either the live picture from the simulator (arm, table, cube) or a computed drawing of the arm in which each link is tinted by how much of its torque limit is in use and an ellipse shows how freely the hand can move	Connects the numbers to what the arm is physically doing; a flat ellipse means near a singular pose
Task panel	Choice of pick style, a Run button, a Stop button, a repeat option, and a status line saying what the simulator is doing and whether the last cube was placed	Lets a user drive the experiment without any technical knowledge

Area	What it shows	Why it matters
Controls	Load (the cube's mass), joint speeds, and a test-fault selector with the joint to affect	Lets the user reproduce the situations the project studies: heavier cube, a worn joint, a delayed command
Main chart	Residual, torque, confidence or payload over the last seconds, for a chosen joint, with the detector's calibrated normal band	Shows the residual as it develops, so a fault is seen as a departure from normal
Residual bars	A bar per joint of the residual as a percentage of the joint's torque limit; click to select a joint	Localises a problem at a glance
Torque breakdown	For a chosen joint, how much torque is gravity, inertia, coupling, friction, and what is unexplained	Explains where the residual comes from
Stat cards	Payload estimate with its uncertainty band, confidence, calibration progress	The headline numbers an operator wants
Alerts and toasts	Plain-language alerts: *unmodelled load on the gripper* or *friction*, which joint, a suggested cause, and a short explanation	Turns statistics into an action
Panels and benchmark tab	Telemetry frame, confidence, payload and about panels, and an objective-6 view comparing the trajectory styles	Depth for the engineer who wants to look further
Guided demo	A scripted tour that changes load, speed and faults in sequence with an explanation of each step	Onboarding for new viewers

9.2 The three data sources

The header offers three sources. **Isaac Sim (live)** connects to the running simulator: the picture is the simulator's own render, the data are its telemetry, and the buttons really move the arm. **Stand-in plant** is a lightweight computed model of the arm that needs no graphics card; it is useful for exploring the interface but is not the simulator and says so. **Replay** loops recorded real simulator logs at zero and one kilogram.

9.3 What happens when you press Run pick

The dashboard sends the chosen style, the cube mass and any fault to the simulator. The simulator places the cube on the table, plans the pick with inverse kinematics in the chosen style, and executes it in real time: it approaches, descends, closes the fingers, lifts, carries, lowers, releases and returns. Throughout, it streams 200 samples per second to the dashboard along with a picture a few times per second. The dashboard restarts its analysis window and detector at the start of each pick, feeds the samples through the same residual, confidence, payload and detection logic described in Section 8, and updates the charts and alerts live. When the pick finishes the simulator reports whether the cube ended within a few centimetres of its target, and the status line shows the verdict.

9.4 Reading a result: three scenarios

- **Healthy pick of 1 kg.** The cube lands within a few millimetres of the target. When the cube is lifted, the dashboard raises *unmodelled load on the gripper*, which is correct: the model has no payload, and the payload estimate rises towards the cube's mass.

- **Pick with friction ×8 on a wrist joint.** The pick still succeeds (the placement error is somewhat larger), but the residual on that joint departs from normal and an alert names the joint.
- **Pick with a locked first joint.** The arm cannot sweep during the carry, the cube ends 15 cm from its target, the pick visibly fails and an alert is raised. The dashboard's live detector currently names the wrong fault type for this case (it reports friction rather than a locked joint); the alert fires but the label needs improvement.

> **Honest note on the live detector**
> The live detector inside the dashboard uses the same decision constants as the offline detector but was tuned on synthetic data. It calibrates on the first three seconds of every pick, so on a healthy pick it can disagree with the offline analysis that uses a healthy reference. Re-tuning it on simulator fault runs is open work.

10. How we know the work is correct

Every claim in this document rests on checks that were designed to catch the project's own mistakes. The main ones are:

- **An independent model.** The analysis model is built separately from the simulator. Agreement between the simulator's own mass, Coriolis and gravity terms and the independent model was measured to better than 0.1 percent on the mass matrix.
- **Automatic run checks.** Every run is rejected unless it passes checks on timing, torque limits, energy consistency, dynamics agreement, speed regime and the correctness of its labels. About 200 runs pass.
- **Matched baselines.** Faulty runs have a healthy twin with the same noise seed, so the only difference is the fault.
- **Seeded noise and false-alarm rates.** Several healthy runs at different seeds give a measured false-alarm rate.
- **Push validation.** A known force is applied to the hand in a soft-control mode, and the estimated force is compared with the truth. With the stricter pre-registered 15 percent criterion the worst case was 15.8 percent (a narrow miss that is explained by joint friction, and that passes the amended criterion); an independent model that includes friction scores a worst case of 9.9 percent.
- **Unit tests.** A large automated test suite covers the geometry of inverse kinematics, the trajectory profiles, the scenario definitions, the detector logic including edge cases found during development, and the dashboard engine.
- **Bisection of surprises.** When old and new data differed, the cause was found by bisection in an older copy of the code: all code was identical, and the difference came from running many scenarios in one long process, which changes the logged gravity term by about a thousandth of a newton-metre. This is documented rather than hidden.

11. What has been achieved

11.1 Capabilities delivered

- A validated simulation of a Franka Panda with correct masses, inertia tensors, friction and torque limits, running under torque control at 1 kHz.
- Real pick-and-place with a physical gripper and cube: in all fifteen physical picks the cube was lifted, carried and placed within about 4 to 10 mm of target.
- Inverse-kinematics-driven tasks in three trajectory styles, plus continuous Cartesian paths.
- A fault injector covering friction, locked joint, delay, backlash, payload change and external push.
- An independent physics model, residual computation, confidence gate, payload estimator and fault detector, tested together on the simulator's data.
- A live dashboard that shows the simulator's picture, runs tasks on command and presents the residual, payload and alerts.
- A single command that starts the simulator and the dashboard together and a second that stops them.

11.2 Results

Task execution. With a real cube, placement error was 3.7 to 4.0 mm at 0.5 kg, 6.2 to 6.4 mm at 1 kg, and 10.2 mm at 2 kg across the three styles. Faults changed what the arm actually does: a seized first joint left the cube 151 mm from target, and a 50 ms command delay made the control loop oscillate, leaving the cube 168 mm away. Friction on a wrist joint left a 16 mm error; friction or a lock on the fourth joint barely mattered because that joint hardly moves on this path.

Payload estimation. Where the mass is known to the physics (modelled), estimates were within 1 to 4 percent. With a real cube the estimates were within 2 to 5 percent at 0.5 and 1 kg, but 25 to 36 percent low at 2 kg.

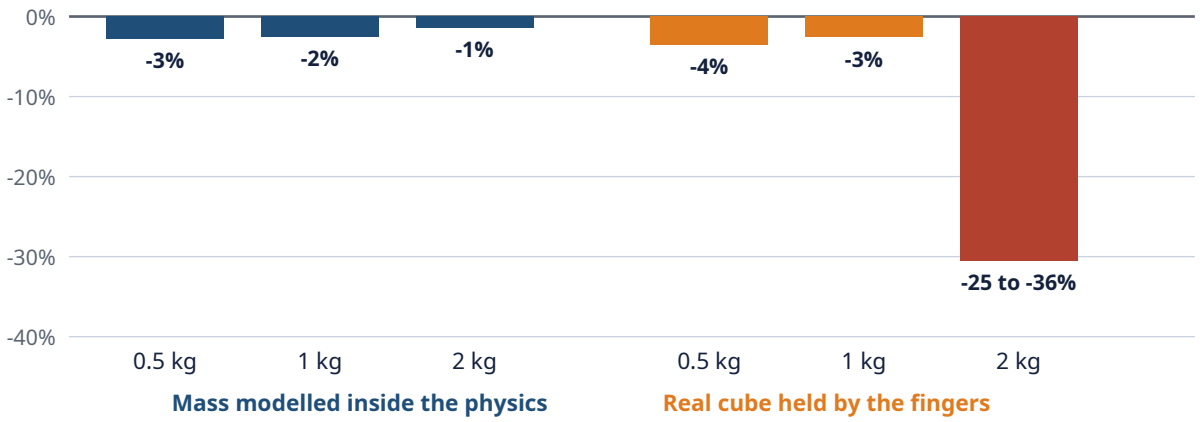


Figure 4 · Payload estimate error by case (midpoint of the observed range; the last bar shows its range).

Fault detection. In the pick runs, with a healthy reference, every injected fault was found on the correct joint: friction ×8 on a wrist joint in 0.05 s, a locked first joint in 0.03 s, friction ×8 and a lock on the fourth joint in about 0.09 s on the real-cube data, and the 50 ms delay in 0.11 s. For the modelled payload the delays were 0.06 to 0.12 s, with the exception of the fourth joint, which hardly moves on this path and was detected only after 3.3 s.

Benchmark of trajectory styles (O6). On a complete pick cycle, the shortest cycle time that keeps peak torque within a budget was measured for each style. Without payload and with a budget of 70 percent of the torque limit, the trapezoidal profile completes in 4.35 s, the S-curve in 5.10 s and the torque-aware profile in 5.64 s; the faster profile costs more peak torque. At 2 kg and the same budget the S-curve (6.09 s) is the fastest of the three that meets the budget, because the trapezoidal profile just exceeds it. No style meets a 40 percent budget, or a 55 percent budget at 2 kg. These fastest runs approach the velocity limit and so are used for time, torque and energy only, never for residual-based inference.

Budget (peak torque)	Payload	Trapezoidal	S-curve	Torque-aware
55%	0 kg	5.67 s	6.09 s	7.54 s
70%	0 kg	4.35 s	5.10 s	5.64 s
70%	2 kg	8.03 s	6.09 s	7.54 s

12. Limitations and open problems

A project review should state what has not been shown. The following limitations are known:

Limitation	Consequence
The payload estimate on a real, contact-held 2 kg cube is 25 to 36 percent low, and became worse when the grip force was doubled	The most realistic case is the weakest. The cube's mass was verified, and friction slip is not the explanation. The cause is not established
A non-repeating task needs a recorded healthy reference for detection	A real deployment must record a healthy run of each task before it can be monitored this way
Friction doubled is not detected; backlash and very short delays are undetectable at the speeds used	There is a floor to what residual-based monitoring can see; it is reported, not hidden
The grasp is open-loop and timed	No force or slip feedback and no re-grasp
Simulation only	Real arms have unmodelled friction, link flexibility, thermal drift and sensor quirks. The numbers show that the method works in a controlled digital twin, not hardware performance
Narrow scope	One arm, one cube, one pick and place location, one table, speeds at or below half the limit
Windowed and headless runs agree to about 0.04 N·m, not exactly; long single-process runs differ by about 0.001 N·m	Regeneration of data is nearly, but not strictly, identical
The dashboard's live detector was tuned on synthetic data and mislabels a locked joint as friction	Alerts fire, but fault classification needs re-tuning on simulator runs
The cube's centre sits slightly closer to the hand than the analysis's payload model assumes	Small in motion, negligible at rest
Simulator and its checks were written by the same team	The strongest independence comes from the separately built analysis model and detector

13. Industrial applications and importance

13.1 Why this matters

Robots are moving from caged, repetitive work into collaborative and flexible tasks, where the arm's behaviour must be trusted without a human watching every motion. At the same time, the cost of an unplanned stoppage in an automated line is high, and failures such as a dropped payload or a seized joint can damage product and equipment or endanger people. A monitoring method that needs no additional sensors and that explains *what* and *where* is attractive because it can be added in software to arms that are already installed, using signals that the controller already has.

13.2 Applications

Application	How the approach helps
Predictive maintenance in manufacturing	A slow rise in a joint's friction residual signals bearing wear or lubrication loss weeks before failure, allowing maintenance to be scheduled instead of suffered
Payload verification in logistics and palletising	Confirm that the correct item, and only one item, has been picked; detect a dropped or mis-gripped item from the sudden loss of load; detect overloaded picks before they exceed safe limits
Collaborative robots near people	An independent check that the arm's physical behaviour matches what its controller believes; unexpected loads or unexpected resistance are flagged. It can complement, not replace, certified safety functions
Semiconductor, pharmaceutical and laboratory automation	Where parts are expensive or contamination is critical, detecting abnormal behaviour at the first sign avoids scrap
Quality assurance of control tuning and motion planning	The trajectory benchmark quantifies the time, peak torque and energy of different motion styles for a given task and load, helping choose settings that protect hardware and save energy
Commissioning and acceptance testing	A digital twin allows engineers to rehearse faults and tune thresholds without risking a real arm
Auditability and compliance	An independent physics check produces evidence that the machine behaved as stated, which supports quality records and incident investigation

13.3 Why a digital twin first

It is not possible to test monitoring methods safely and repeatably on real arms: nobody wants to seize a bearing on purpose, run a hundred healthy cycles with controlled noise, or apply exactly 3 N of side force. The simulator provides unlimited, exactly labelled examples, which allows false-alarm rates and detection delays to be measured, thresholds to be chosen on evidence, and failure modes to be discovered cheaply before any hardware is involved. The price is the gap between simulation and reality, which is why the limitations are stated up front and why the next step in any serious programme is validation on a physical arm.

13.4 Path to deployment

- Re-tune the dashboard's live detector on the simulator's fault data and add a proper classification of fault type.
- Investigate the underestimate of heavy, contact-held payloads, for example by considering the dynamics of the cube during contact, and decide whether the cause is in the estimator's model

or in the simulation.

- Add feedback to the grasp (force and slip) and examine regrasping.
- Record healthy reference runs of each production task and package the monitor as a service that consumes controller telemetry.
- Validate on a physical Franka Panda: replace the simulator by the real controller's telemetry, identify friction and noise levels from data, and re-measure detection delays and false-alarm rates.
- Extend to faster motion by modelling the drive effects that currently limit the trusted speed regime to half the velocity limit.

14. Using the system

A single launcher starts everything. Starting it brings up the simulator without a window and the dashboard in a browser; the simulator's picture appears inside the dashboard. The first start takes a few minutes while graphics shaders compile; the dashboard says it is waiting until the arm is ready. The user then chooses a pick style, optionally a cube mass and a fault, and presses Run pick. A second command stops both parts. Optional modes open a separate simulator window, run the dashboard alone on its stand-in model, report what is running, show the logs, or run the full simulator test suites followed by the automatic checks.

A typical session for a newcomer: start the system; run a healthy pick with the default 1 kg cube and watch the cube being lifted and placed while the load alert appears and the payload estimate rises; run the same pick with friction on a wrist joint and compare the residual bars; run it with a seized first joint and watch the pick fail; then use the guided demo for a narrated tour.

15. Glossary

Term	Meaning
Digital twin	A simulation detailed enough to stand in for a real machine, used here as a source of exactly labelled data
Plant	The machine being controlled or monitored: here, the simulated arm
Joint torque	The rotational force a motor applies at a joint, in newton-metres
Inertia / mass matrix	How hard it is to accelerate the arm in each direction at the current pose
Coriolis and centrifugal terms	Torques that arise because links move while other links rotate
Computed-torque control	A controller that uses the arm's model to cancel its own dynamics and add feedback corrections
Residual	Applied torque minus the model's predicted torque; what the model cannot explain
Jacobian	The matrix that converts joint velocities into hand velocities, and by transpose hand forces into joint torques
Singular pose	A configuration in which the arm cannot move or resolve force in some direction; estimates become unreliable
Inverse kinematics	Finding joint angles that place the hand at a desired position and orientation
Redundant degree of freedom	An extra joint beyond what is needed to position and orient the hand, giving many solutions

Term	Meaning
Tare	Subtracting the reading obtained under known conditions (empty gripper) to remove constant bias
Tracking error	Commanded position minus actual position
False alarm	An alert raised when nothing is wrong
Backlash	Play in a gear train that lets the output move without the input moving
Jerk	The rate of change of acceleration; smaller jerk means smoother motion
Open-loop	Acting on a schedule without checking the outcome